Large Language Models for Software Requirements Classification: Cross-Dataset Generalisation and Cost-Efficiency

Fatma El-Zahraa Samir, Khaled T. Wassif, and Lamia AbouZeid
Faculty of Computers and Artificial Intelligence, *Cairo University*, Giza, Egypt
fatmaelzahraasamirabdelfattah@gmail.com

***Abstract*—Recent work reports that prompting a large language model (LLM) classifies software requirements about as well as fine-tuning a small transformer, and that labelled training data may therefore no longer be needed. Nearly all of that evidence is collected in one place: the model is fitted, or its examples chosen, on the corpus that also supplies the test items, and what the technique costs to run is rarely reported. This paper repeats the comparison with both gaps closed. Fine-tuned encoders and prompted LLMs from three access tiers classify the same requirements from two public corpora, under the same tasks and the same scoring rule, at three levels of distribution shift: inside one corpus, across held-out projects, and across a corpus annotated by a different team. Every encoder–LLM comparison is paired on identical items and corrected for multiple testing as a single family. The ranking of the two approaches is not stable across those levels: inside a corpus the encoders lead and are never significantly beaten, but once the test corpus is annotated by someone else the ordering reverses. The reason is not that the transferred encoder fails to recognise the new vocabulary; it carries over the class balance of the corpus it was trained on, so it goes on finding the rare class far too often. The prompted models have the same weakness from a different source, since their class balance is set by the wording of the instruction. The encoders nevertheless remain an order of magnitude cheaper per item and repay their one-off training after a few hundred classifications. The evaluation regime therefore belongs in the claim: a score obtained in-distribution does not predict which approach wins once the corpus changes, and can point the wrong way.**

***Index Terms*—requirements classification, large language models, fine-tuning, generalisation, distribution shift, empirical software engineering, cost-efficiency**

I. Introduction

Requirements classification assigns a natural-language requirement statement to a category, such as functional (FR) against non-functional (NFR), or security-relevant against not. ISO/IEC/IEEE 29148 defines requirements and their attributes across the life cycle [2], and classification is one of the first analytical steps after elicitation. Doing it well supports early risk identification and architectural choice; doing it badly propagates downstream as rework [1]. Two decades of automation moved the task from weighted indicator terms [1] through supervised learning on engineered features [3] to fine-tuned transformers [4], and since 2024 to prompted generative LLMs [6], [9]–[11], [14]; the systematic mapping of Kaur and Kaur documents the same progression [7].

It is worth being precise about what that literature claims, because the motivation for this study is often summarised too strongly. We are not aware of any paper stating that requirements classification is solved. What several report is parity or better between prompting and supervision on the corpora they use: Binkhonain and Alfayaz find that few-shot prompting matches or exceeds a fine-tuned transformer and conclude that prompting is viable where labelled data is scarce [9]; Tang et al. report an 8 B open model passing a fine-tuned baseline on PROMISE [11]; and Alhoshan et al. report that generative models perform competitively without task-specific training [10], extending earlier evidence that zero-shot embedding methods had already made labelled data non-essential [5]. Together these make "prompting can replace fine-tuning" a reasonable reading of the state of the art, and it is that reading, rather than any individual paper's claim, that this study puts to the test.

The reading rests on a measurement convention. Almost every published result is obtained in-distribution: the model is fine-tuned, or its exemplars selected, on the corpus that then supplies the test items. Where prior work reports improved generalisability it usually means transfer across *tasks* on the same corpora rather than to unseen projects or an independent dataset [9] — the weakness NoRBERT was built to address, since it was motivated by classifiers over-fitting project-specific wording [4]. A second omission compounds the first: inference cost, latency and model size are seldom quantified, although they decide whether a technique can be adopted at all without a commercial API budget.

This paper asks what happens when both omissions are repaired. We hold the corpus, the items, the label sets and the scoring rule fixed, and vary only the distance between the data a model was fitted on and the data it is scored on. We contribute a paired, multiplicity-controlled benchmark of twelve configurations over three tasks, two corpora and three regimes; evidence that the ranking depends on the regime; a diagnosis of the cause as a mis-set class prior that both families share; and an accuracy–cost comparison priced from the runs themselves. **RQ1**: how do open and commercial LLMs compare with fine-tuned encoders within one corpus? **RQ2**: how much is lost under cross-project and cross-dataset evaluation, and does the loss differ between the families? **RQ3**: what is the accuracy-versus-cost trade-off, and what is the cheapest freely available option that is good enough?

II. Related Work and the Evaluation Gap

A. From indicator terms to prompting

Rule-based work established the task and produced the PROMISE NFR collection [1]; classical supervised learning followed, with SVMs separating FRs from NFRs [3]. Transfer learning then reset the state of the art: NoRBERT fine-tunes BERT and reports macro-F1 in the mid nineties on the PROMISE categories [4], a pattern the systematic mapping of Kaur and Kaur later confirmed [7]. Zero-shot embedding methods showed labelled data was not strictly necessary [5], and prompted generative LLMs now dominate.

B. Recent LLM studies, 2024–2026

Binkhonain and Alfayaz [9] report the study closest to ours: five LLMs under four prompting strategies over three tasks on PROMISE and SecReq, against a fine-tuned transformer that few-shot prompting matches or exceeds. Alhoshan et al. [10] ran more than 400 experiments over three open generative models, and Tang et al. [11] identified an over-prompting effect in which extra in-context examples degrade accuracy. Elsewhere the task has been compared across classical and generative techniques [6], embedded in a systems engineering workflow [8], extended to app-store reviews [12], run against GPT-4o [13] and benchmarked against expert judgment [14].

C. What this literature does not measure

Table I summarises the evaluation coverage of those studies along the axes that decide whether a result transfers to practice, and three patterns recur. First, evaluation is in-distribution: the same corpus supplies the training data or the exemplars and the test items. Only the app-review study [12] tests genuinely different text, and it reports markedly lower precision on several categories, a signal the literature has not followed up. Second, cost is not reported, even in the studies that use open models [10], [11]. Third, the same few small, ageing, public corpora underlie nearly every result, so contamination of LLM pre-training data is acknowledged but not addressed [27]. These gaps interact: if in-distribution parity is what motivates replacing supervision with prompting, and is also what corpus reuse and contamination would produce spuriously, then the comparison that settles the question has not yet been run.

TABLE I
Evaluation coverage of recent LLM-based requirements-classification studies. ✓ = reported; ✗ = not reported; ∼ = partially. "Cross-project" means held-out unseen projects; "cross-dataset" means an independently produced corpus. The table describes prior work only.

Study	Open LLMs	Comm. LLMs	Cross-project	Cross-dataset	Cost
El-Hajjami et al. [6]	✗	✓	✗	✗	✗
Peer et al. [8]	✗	✓	✗	✗	∼
Binkhonain & Alfayaz [9]	✓	✓	✗	✗	✗
Alhoshan et al. [10]	✓	✗	✗	✗	✗
Tang et al. [11]	✓	✓	✗	✗	✗
Chaudhary et al. [12]	✗	✓	✗	∼	✗
Vasudevan et al. [13]	✗	✓	✗	✗	✗
Levy et al. [14]	✓	✓	✗	✗	✗

III. Study Design

Twelve model configurations classify the same requirements, under the same tasks, against the same frozen splits, and are marked by one rule, so that every comparison is paired on identical items (Fig. 1). The pipeline runs in seven stages, each writing an artefact the next one reads, and every number in Section IV is read back out of those artefacts by the scripts that typeset the tables and figures.

A. Data preparation

PROMISE_exp [16], an expanded release of the PROMISE NFR collection, contributes 968 requirements from 47 projects, each labelled FR or NFR and, when NFR, with one of eleven quality sub-types. SecReq [15] contributes 444 sentences from 3 industrial security specifications, each labelled security-relevant or not.

Preparation happens once, in the first stage. Texts are normalised and the label strings mapped onto one controlled vocabulary, so both corpora express the security concept in the same words. Exact duplicates are removed — 67 rows, all within a corpus rather than across the two — taking the raw 969 and 510 rows down to 1,412 requirements in 50 project groups. The 14 split families, 107 folds in all, are generated once with seed 42 and stored, keyed to identifiers pinned to the SHA-256 hash of each source file; grouped-fold membership is library-version dependent, so later stages read that file and never regenerate it.

Two properties of the prepared data matter throughout. The first is a difference in class balance: security requirements are 12.9% of PROMISE_exp but 39.9% of SecReq, a threefold difference in prevalence for a concept both corpora annotate independently, and the cross-dataset regime is built on exactly that contrast. The second is that the security label reaches the two corpora by different routes. In PROMISE_exp it comes from the security sub-class of the NFR taxonomy, so every PROMISE_exp item labelled security is also NFR and none of the 444 FRs is; in SecReq it applies to any security-relevant sentence, and there is no FR/NFR annotation at all. The cross-dataset regime therefore carries a shift in definition alongside the shift in distribution (Section VI). Because security is the only task both corpora annotate, it is also the only task on which a cross-dataset comparison can be posed at all.

B. Tasks, label sets, and why they are not staged

Binary FR/NFR uses the native PROMISE_exp labels over all 968 PROMISE_exp items. Binary security is native in SecReq and derived in PROMISE_exp, which gives the same task on two independently produced corpora. NFR sub-type classification uses the eleven-class taxonomy of Cleland-Huang et al. [1], whose categories align with the ISO/IEC 25010 quality characteristics [21].

The three tasks are evaluated independently, and the pipeline is not staged. The security classifier is never shown the output of the FR/NFR classifier; every item is presented to it directly, including the 444 PROMISE_exp items labelled FR. We

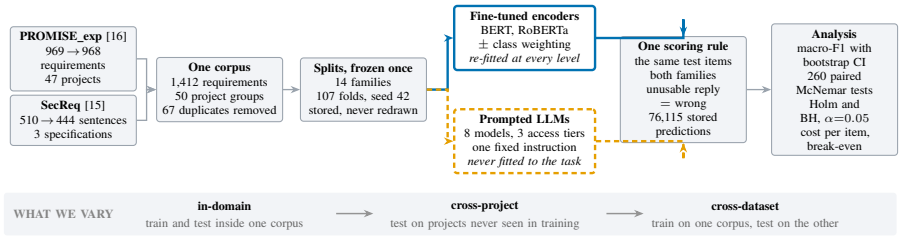

Fig. 1. How the experiment is put together. One data path on the left feeds two families of model that differ in a single respect: whether they may learn from labelled training data. The encoders (solid, upper) read the training folds and are re-fitted at each level of the band beneath, so their scores move; the prompted models (dashed, lower) read only the test items, so their scores cannot move with the level, which makes them a fixed reference. Both families answer the *same* items — which is what makes every encoder–LLM comparison a paired one — and both are marked by the same rule. The band is the only thing we deliberately vary.

say this explicitly because the PROMISE_exp annotation is hierarchical, security being a sub-class of NFR, so it would be natural to assume the pipeline is too. It is not, and deliberately so: a staged design would make the security score depend on the errors of the FR/NFR classifier, and it would have no counterpart in SecReq, where no FR/NFR labels exist.

Several sub-type classes have single-digit support, so that task is evaluated at three granularities: eleven classes (524 items), the six most frequent (433) and the four most frequent (353). For an encoder the label set fixes the size of the classification head, so the model is re-trained for each granularity. A prompted model has no head to resize and no training at all, so it is fair to ask what top-4 and top-6 can mean for it. They change three things: the candidate labels are listed inside the prompt, so at top-4 the model is offered four and at all-11 eleven; the evaluation frame changes, since an item enters the top-4 frame only if its true label is one of those four; and macro-F1 is computed over the full declared label set, so a model is penalised for every declared class it never predicts. Granularity is a property of the question and of the scoring, not of the model weights, which is why it is measurable for both families.

C. The three evaluation regimes

The 14 frozen split families implement three levels of increasing distribution shift, and this is the study's independent variable. *In-domain* is stratified 5-fold cross-validation within one corpus, the setting essentially all prior work reports; it asks how well the model does on more of the same data. *Cross-project* uses grouped folds that hold out entire project groups, plus a leave-one-project-out variant over the 47 PROMISE_exp projects; it asks how well the model does on a new project inside a familiar corpus. *Cross-dataset* fits on the security task of one corpus and tests on the other, in both directions, with an identical label set but a different annotation provenance; it asks the question a practitioner actually faces, which is how well the model does on requirements written and labelled by someone else.

The asymmetry between the families is the analytical lever. The encoders are re-fitted at every regime on that regime's training folds, so their scores move; the prompted models have no training distribution to shift, so their scores cannot move with the regime, which is why a single value is written across the regime columns of Table III for each of them. That fixed reference is what lets the encoders' loss be attributed to the shift rather than to the difficulty of the items. Every requirement is a test item exactly once within each encoder family; the prompted models answer stratified samples of the same items (300 per binary cell, 200 per sub-type cell, in proportion to the class prior with a floor of ten per class), and the six local models additionally answer the full frames.

D. The two model families, and why they are compared

Table II lists the twelve configurations, and the architectural difference between them is not incidental to the question. A fine-tuned encoder is a bidirectional, encoder-only transformer, pre-trained with masked language modelling and then adapted by supervised training of a classification head over the pooled sentence representation, so both its decision boundary and its class prior are learned from the labelled training folds. A prompted LLM is a decoder-only autoregressive model never adapted to the task: it receives the label vocabulary as text, answers in generated tokens, and its implicit class prior comes from pre-training and from the instruction rather than from any corpus of requirements. That contrast is what makes the comparison informative, because the two families should react differently to a shift in distribution precisely by acquiring their prior in different places, and Section IV-C shows that they do. Prior work has argued that architecture affects results on this task [13], so we treat it as a declared property and report every result per configuration.

We fine-tune `bert-base-uncased` [18] and `roberta-base` [19] with a classification head (AdamW, lr 2×10^{-5}, batch 16, max length 128; 3 epochs on the binary tasks, 10 on the sub-types), each with inverse-frequency class weighting, plus an unweighted control on the security task. The local tier runs six instruction-tuned models of 2.6–8.0 B parameters on a single free-tier NVIDIA T4 under 4-bit NF4 quantisation [20]; the hosted and commercial tiers reach Llama-3.3-70B and Gemini 3.1 Flash-Lite through free API tiers. A ninth prompted model failed the gate of Section III-F and enters no table.

E. The prompt, and why it is fixed

Each prompted model answers a fixed instruction that names the task, lists the label vocabulary and requires exactly one

TABLE II
THE TWELVE MODEL CONFIGURATIONS. "B" IS THE PUBLISHED PARAMETER COUNT OF THE EXACT CHECKPOINT, IN BILLIONS; GEMINI'S IS UNDISCLOSED AND IS RECORDED AS UNKNOWN RATHER THAN ESTIMATED.

Configuration	Checkpoint / endpoint	B	Access
BERT (w / u)	`bert-base-uncased`	0.11	fine-tuned
RoBERTa (w / u)	`roberta-base`	0.13	fine-tuned
Gemma-2-2B	`gemma-2-2b-it`	2.61	local, 4-bit
SmolLM3-3B	`SmolLM3-3B`	3.08	local, 4-bit
Qwen2.5-3B	`Qwen2.5-3B-Instruct`	3.09	local, 4-bit
Phi-4-mini	`Phi-4-mini-instruct`	3.84	local, 4-bit
Qwen2.5-7B	`Qwen2.5-7B-Instruct`	7.62	local, 4-bit
Llama-3.1-8B	`Llama-3.1-8B-Instruct`	8.03	local, 4-bit
Llama-3.3-70B	`llama-3.3-70b-versatile`	70.6	hosted
Gemini 3.1 F-Lite	`gemini-3.1-flash-lite`	n/a	commercial

word in reply, sent as a single user message. Decoding is greedy, temperature 0 with at most twelve new tokens, so the only stochastic element left is which items are sampled. The base FR/NFR instruction is reproduced below; the sub-type tasks use the same template with the permitted labels listed in place of "FR"/"NFR", which is how the top-4, top-6 and all-11 variants differ for a prompted model.

```
You are classifying a software requirement.
Decide whether it is a FUNCTIONAL requirement (FR) --
something the system must DO -- or a NON-FUNCTIONAL
requirement (NFR), a quality, constraint or property
such as performance, security or usability.
Answer with exactly one word: "FR" or "NFR".
⟨few-shot only: k exemplars here⟩
Requirement: """the requirement under test"""
Answer:
```

Fixing the instruction needs justifying, because a different wording could give a different result and a better one almost certainly exists. The template follows the zero-shot format used by the studies we compare against [9]–[11]: a task statement, a list of permitted labels, and an instruction to answer with the label alone. We did not tune it against the test items. The design requires a *fixed* instruction so that the only quantity varying across the three regimes is the distribution shift; re-optimising the wording per regime would make any difference between regimes unattributable to the shift, which is the one thing this study sets out to measure. The prompt is a control, not a recommendation.

What a better wording would be worth is measured rather than assumed. A sub-study re-runs two models over three phrasings of the same question (base, terse and verbose) with the answer contract held identical, over 9 paired cells. An oracle picking the best wording per cell would gain a median of 0.003 macro-F1 and at most 0.036 — far less than the differences our conclusions rest on, and picking it would require labelled data from the target corpus, the very resource prompting is meant to do without. A second sub-study is a few-shot ladder ($k \in \{2, 4, 8\}$ on the binary tasks, $k \in \{1, 2\}$ per class on the sub-types) with class-balanced exemplars carved out before the evaluation frames are formed, so an exemplar is never also a test item.

F. Scoring, and how to read the numbers

Replies are parsed by a strict, word-boundary-aware rule: a leading label wins; otherwise exactly one label named anywhere in the reply wins; a reply naming both labels or neither is unusable and is scored as wrong. This is harsher than the lenient protocols common in prior work, and it matches deployment, where an unusable answer is not a free pass. The rule runs at generation time and again in an idempotent repair pass over the archived outputs, so stored and scored predictions cannot drift apart. Encoder rows are argmax predictions and are exempt from the parse-rate gate.

Four kinds of quantity appear in this paper. They are easy to confuse, so we say what each one is and name it explicitly at every use.

- **macro-F1**: the unweighted mean of the per-class F1 scores over the full declared label set, on a 0–1 scale. It is primary because it does not let a model score well by ignoring a rare class; the majority-class baseline in the last row of Table III gives it a floor.
- **Relative loss** ($\Delta\%$): the share of a configuration's *own* in-domain macro-F1 that it loses in a shifted regime, on the same items. Only the encoders have it, because only they have a fit to lose.
- **Predicted class share**: the fraction of test items a model assigns to a class. Not an accuracy — it is what the model takes the class balance to be, read against the true prevalence in the same items.
- **Test counts**: numbers of statistical comparisons, not of requirements. One unit is one exact McNemar test between one encoder and one prompted model in one cell.

Uncertainty uses a stratified bootstrap of 2,000 resamples within each true class (seed 42). Encoder–LLM comparisons use the exact two-sided McNemar test on paired disagreements [22], and all 260 are corrected as one family under Holm [23] and Benjamini–Hochberg [24] at $\alpha = 0.05$, with verdicts keyed on the BH-adjusted p (175 significant before correction, 170 after BH, 131 after Holm). A cell — one model, task, dataset and regime — is reported only if every class has at least five test items, the cell holds at least 40 items, and the model parsed at least half of its responses; 6 of 87 cells fail that gate and are excluded rather than imputed. The store holds 76,115 rows.

G. How cost is computed

Every prediction row records token counts, wall-clock latency and, for the encoders, training time, so cost is computed from recorded quantities rather than estimated after the fact. The basis differs by tier, because the two kinds of model are billed differently.

Models that run on our own hardware are priced on compute. That covers the encoders and all six local open-weight models: they are free to download and carry no licence fee, so the only thing to charge for is the machine time they occupy. We price it as an imputed rental of the benchmark GPU — an AWS g4dn.xlarge with one NVIDIA T4, at the published on-demand

list rate for us-east-1 of $0.5260 per hour, recorded in the manifest with the date it was checked (AWS EC2 g4dn.xlarge (1x T4), us-east-1, verified 2026-08-06). That is a list price for hardware, not for the model, and it assumes unbatched inference; the stored sensitivity table reports the same costs at the spot rate ($0.3162/h) and at batch sizes 8 and 32, which cut the local figures by roughly an order of magnitude.

Models behind an API are priced on tokens, at the provider's published list rate per million input and output tokens: $0.25/$1.50 for Gemini 3.1 Flash-Lite and $0.59/$0.79 for Llama-3.3-70B, taken from the providers' public rate cards in August 2026 and frozen into every prediction row at generation time, so a later price change cannot alter a stored result. Both were in fact run on free tiers; the list rate is what the same volume would have cost.

Neither basis is a fixed property of the technique, and cost should be expected to move with model size and capability. Within the compute-priced tier it plainly does: over the 10 locally executed configurations, cost per item rises with parameter count (Spearman $\rho = 0.85$). That is why Section IV-D states the comparison as a ratio between tiers rather than an absolute price that would date. Encoder cost separates the one-off training cost C_0 from the marginal per-item cost c_{enc}, giving a break-even volume against any prompted alternative of marginal cost c_{alt}:

$$N^{\star} = \frac{C_0}{c_{\text{alt}} - c_{\text{enc}}} \tag{1}$$

Because free-tier latency is dominated by queueing on the provider's side, wall-clock projections use the per-request median latency, not the mean.

IV. Results

Table III is the paper's single results table: all twelve configurations, the six task-by-corpus cells and every regime each supports, so that encoders and prompted models are read off the same rows and columns. The encoders have one score per regime because they are re-fitted at each; a prompted model has one score per cell, written across the regime columns it spans, because a model that consumes no training data cannot score differently when that data changes.

A. RQ1: within one corpus, supervision is still ahead

Reading the ID columns of Table III, the fine-tuned encoders hold the best score in five of the six cells: class-weighted BERT on FR/NFR and on PROMISE_exp security, class-weighted RoBERTa on all three sub-type granularities. The exception is SecReq security, where 4-bit Qwen2.5-7B posts the better point estimate (0.846 against 0.836), a lead that does not reach significance (exact McNemar $p = 0.696$; BH-adjusted 0.774).

Three observations qualify that ranking. Scale does not order the results: the 2.6 B Gemma-2-2B scores 0.900 on PROMISE_exp security against 0.891 for the hosted 70 B model. The encoder advantage widens as the label set grows, from 0.014 macro-F1 on top-4 to 0.095 on all-11 over the best prompted model, as expected if supervision mainly buys

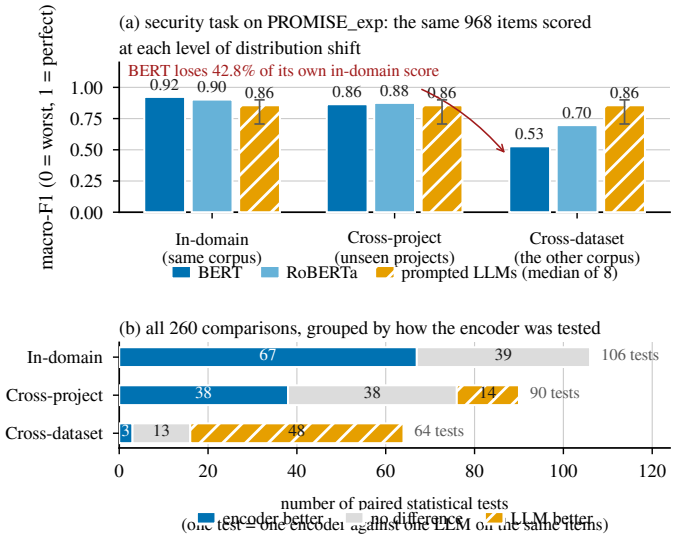

Fig. 2. (a) Security classification on PROMISE_exp. All bars are scored on the same 968 items at every level; the encoders are re-fitted at each level, while the third bar (median of the eight prompted models, whisker = their full range) is flat because those models are never fitted. The red annotation is a *relative loss of macro-F1* — BERT's cross-dataset score is 42.8% below its own in-domain score — not a share of items. (b) The 260 paired comparisons, grouped by the regime the encoder was evaluated under. One unit on the axis is one McNemar test, not one requirement.

the long tail of rare classes. And because cell sizes differ, the top-4 ranking was recomputed on the 59 items every model answered; the best encoder still leads there by 0.036 macro-F1, so the ordering is not an artefact of coverage. Over the 106 in-domain comparisons the encoders win 67 and lose none, the remaining 39 being inconclusive (Fig. 2(b)).

B. RQ2: the ranking reverses under distribution shift

Cross-project transfer is survivable. Holding out unseen projects costs the encoders 3.0–11.9% of their in-domain macro-F1 on PROMISE_exp, and at most 29.6% anywhere in this regime (BERT on SecReq, whose three project groups make a held-out group a third of the corpus). They stay ahead or tied in 76 of the 90 paired tests: the degradation NoRBERT anticipated [4], real but moderate.

Cross-dataset transfer reverses the ranking. The class-weighted BERT that scores 0.923 on PROMISE_exp security falls to 0.528 on the same items once its training corpus is swapped for SecReq. That is 42.8% of its in-domain macro-F1, the largest relative loss in the study; it belongs to the cross-dataset regime and should not be read against the cross-project figures above, whose largest value is 29.6%. RoBERTa is more robust (-22.7%), the unweighted controls behave like their weighted counterparts (-41.1% and -21.8%), and the reverse direction is milder still (-15.5% to -17.4%). The prompted models, having nothing to re-fit, move only with the difficulty of the corpus. Over the 64 cross-dataset comparisons they win 48 and the encoders 3: the in-domain tally of 67–39–0 becomes 3–13–48, on the same twelve models and the same scorer.

TABLE III

MACRO-F1 FOR ALL TWELVE CONFIGURATIONS, ACROSS EVERY TASK, CORPUS AND REGIME. ID = IN-DOMAIN, XP = CROSS-PROJECT, XD = CROSS-DATASET. BEST PER COLUMN IN BOLD; "–" MARKS A CELL BELOW THE REPORTABILITY GATE OF SECTION III-F, OR A REGIME THAT DOES NOT APPLY. A PROMPTED MODEL'S SCORE IS WRITTEN ONCE ACROSS THE REGIMES OF ITS CELL, BECAUSE IT CONSUMES NO TRAINING DATA AND THE REGIME CANNOT MOVE IT. ENCODER CELLS ARE CROSS-VALIDATED OVER ALL ITEMS ($n = 968/444/968/353/433/524$ BY TASK GROUP); PROMPTED CELLS ARE STRATIFIED SAMPLES OF THEM ($n = 59$–960).

	Security — PROMISE_exp			**Security — SecReq**			**FR/NFR — PROMISE_exp**		**Sub-types top-4**		**Sub-types top-6**		**Sub-types all-11**	
Configuration	ID	XP	XD	ID	XP	XD	ID	XP	ID	XP	ID	XP	ID	XP
BERT_{w}	**0.923**	0.865	0.528	0.836	0.588	0.690	**0.908**	0.843	0.878	0.800	0.827	0.735	0.657	0.579
BERT_{u}	0.907	–	0.534	–	–	0.702	–	–	–	–	–	–	–	–
RoBERTa_{w}	0.902	0.876	0.698	0.824	0.668	0.696	0.896	0.853	**0.936**	0.835	**0.837**	0.753	**0.724**	**0.688**
RoBERTa_{u}	0.903	–	0.707	–	–	0.678	–	–	–	–	–	–	–	–
Gemini 3.1 Flash-Lite		0.829			0.713			0.867		0.922		–		–
Llama-3.3-70B		0.891			0.814			0.852		0.907		0.782		–
Qwen2.5-7B		0.884			**0.846**			0.656		0.876		0.824		0.576
Llama-3.1-8B		0.872			0.845			0.682		0.700		0.717		0.629
Gemma-2-2B		0.900			0.821			0.663		0.760		0.603		0.527
Phi-4-mini		0.798			0.635			0.727		0.727		0.712		0.599
SmolLM3-3B		0.841			0.717			0.562		0.763		0.693		0.482
Qwen2.5-3B		0.707			0.741			0.554		0.732		0.689		0.543
majority-class baseline		0.466			0.376			0.351		0.131		0.075		0.035

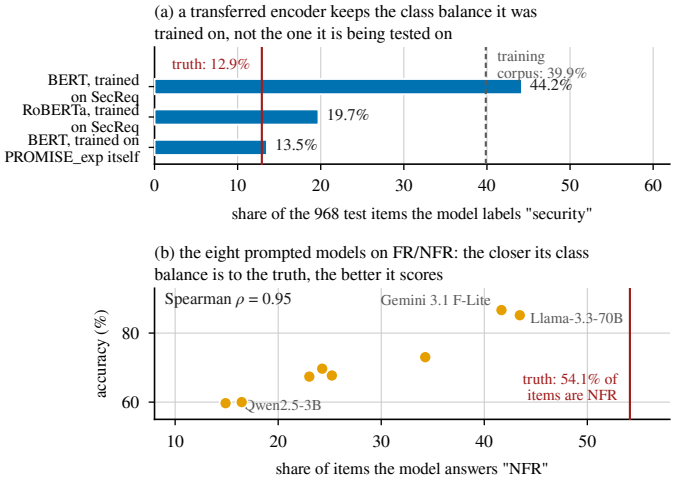

Fig. 3. One failure mode reached by two routes. (a) The share of the 968 PROMISE_exp test items each encoder labels *security* — a share of items, not a score. Red line: the true share in those items. Grey line: the share in SecReq, the corpus the transferred models were trained on. (b) The eight prompted models on FR/NFR; the horizontal axis is again a share of items and each point is one model.

Corpus-level averages hide per-project failure. On the leave-one-project-out variant of FR/NFR, over the 37 projects with comparable coverage, every configuration scores 0.000 on at least one project and 1.000 on another. Only the middle separates them: the encoders' median per-project accuracy is 0.917 against 0.545–0.750 for the small open models.

C. The cause is a mis-set class balance, in both families

A drop in score does not by itself say why a model failed, and the two candidate explanations imply different remedies. If the transferred encoder fails because the new corpus uses unfamiliar vocabulary, the fix is more or broader training text; if it fails because it has the wrong idea of how common the class is, the fix is a prior correction, which needs no labels from the target corpus at all. The per-class evidence separates the two (Fig. 3).

After transfer from SecReq, BERT labels 44.2% of the PROMISE_exp items *security*, against a true prevalence of 12.9% in those items and 39.9% in the corpus it was trained on. That 44.2% is a predicted class share, not a loss of score; its closeness to the 42.8% quoted above is a coincidence of arithmetic, and the two must not be read together. The consequence is visible per class: precision on the security class falls from 0.847 to 0.210 while recall falls far less, from 0.888 to 0.720. The model still finds the security requirements; what it cannot do is stop finding them, which is the signature of a shifted prior rather than a failure to read the text. The picture is not uniform — RoBERTa carries less of its source prior across (19.7% against 12.9% true) and loses correspondingly less — but the dominant term in the largest observed collapse is a prior the model brought with it, which is an encouraging diagnosis: prior shift is correctable without labels from the target corpus [25].

Turning the same instrument on the prompted models finds the same defect arriving by a different route. On FR/NFR the true NFR share is 54.1%, and every prompted model predicts NFR less often, from 14.9% (Qwen2.5-3B) to 43.4% (Llama-3.3-70B). Their accuracies rank almost exactly with that miscalibration (Spearman $\rho = 0.95$, Fig. 3(b)), so the weak FR/NFR scores of the small open models reflect a systematic bias towards answering FR, not an inability to read the requirement.

The wording of the question moves that balance directly, which makes the link causal rather than merely correlational. Across the 9 paired cells of the phrasing sub-study the median spread between the three wordings is 0.044 macro-F1, but the largest is 0.485, for Gemma-2-2B on SecReq security: the terse variant is the only one phrased as a leading yes/no question, and under it Gemma-2-2B answers *security* for 96.7% of SecReq items against a true rate of 39.7%. This is not a

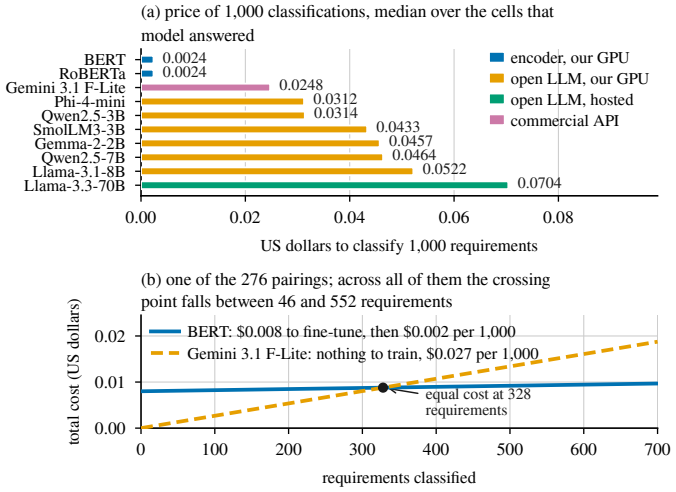

Fig. 4. (a) Price of 1,000 classifications, on the basis of Section III-G: compute time for anything run on our own GPU, published token rates for the two API models. (b) Total cost against volume for one encoder–LLM pair. The encoder starts above zero because it must be trained once; the prompted model starts at zero and rises faster. The crossing point is $N^\star$ from (1).

formatting artefact, since the parser accepted 99.83% of the sub-study's 12,042 responses. Prompt sensitivity is itself well documented [26]; what is new is that it lands on the same failure mode as the encoders' transfer loss. Few-shot exemplars help only where they carry information about the class balance: over the 90 paired comparisons the median gain is +0.024 macro-F1, negative on the binary tasks (mean −0.007) and positive on the sub-types (mean +0.062), the over-prompting effect Tang et al. report [11]. This is one defect with two sources: the task needs a class prior that cannot be read off the requirement text, and each family takes one from somewhere it should not.

D. RQ3: the accuracy–cost trade-off

Encoder inference costs $0.0024 per 1,000 classifications on the imputed T4 rental of Section III-G. The prompted configurations range from $0.0248 to $0.0704 per 1,000 on the same basis, taking each model's median across its cells: 10 to 29 times the encoder, about 19 times at the medians (Fig. 4(a)). One-off fine-tuning is small, a median of $0.0089 per deployable model (172 GPU-seconds over all folds of a family), so the break-even of (1) arrives quickly: over all 276 encoder–LLM pairings $N^\star$ lies between 46 and 552 items, median 215. In concrete terms, classifying 10,000 requirements costs $0.03 with a fine-tuned encoder, $0.25 with the commercial API and $0.70 with the hosted 70 B model. The Pareto frontier holds 10 cells, 9 of them encoders; the one prompted frontier point is 4-bit Qwen2.5-7B on SecReq security, which buys 0.010 macro-F1 for roughly 19× the cost per item.

Two qualifications keep these figures in proportion: they are ratios measured on one machine at one date, and the model prices computation only, so the annotation an encoder needs is excluded, which favours the encoder. Latency behaves differently — 0.016 s median per request for the encoders, 0.095 s for the hosted 70 B model, 0.194–0.356 s for the local open models and 0.614 s for the commercial API.

V. Discussion

A. Generalisability

The most consequential result is not that one family is better than the other, but that the answer to "which is better?" changes between two protocols the literature treats as interchangeable, on the same corpora, items and scorer. A benchmark reporting only in-distribution numbers is not a weaker version of one that reports transfer; on this evidence it can report the opposite ordering. Two things follow: the regime belongs in the claim, next to the metric; and "generalisation" should mean held-out projects or an independently annotated corpus, not transfer across tasks within one dataset [9]. The limits of our own claim are equally explicit: we observe the reversal on one corpus pair, one task and one language, so the study shows that it is possible and large when it happens, not that it occurs at any particular rate. The mechanism we identify is what would make it recur, and it is checkable on any new pair without re-running our models — compare the share of items a model assigns to the rare class against the prevalence of that class in the target data, and if the two disagree badly the score will follow.

B. Consistency and reliability

Accuracy averaged over a corpus is not the only property a practitioner needs, and two kinds of instability appear here, one per family. The encoders are stable given a fixed training distribution and unstable when it moves: their scores change little across folds inside a corpus, but they lose up to 42.8% of their in-domain macro-F1 when the training corpus is replaced. That failure is at least predictable before deployment, from the class balance of the corpus they were fitted on relative to the target. The prompted models are stable across regimes by construction but unstable with respect to the question: re-wording the same instruction, with the answer contract held fixed, moved macro-F1 by up to 0.485 in one cell. Pinning the prompt gives repeatable answers, but the level at which they repeat was set by a wording choice no in-distribution study would surface. Both families also show wide per-project variance.

C. The accuracy–cost trade-off in practice

For a team with a few hundred labelled requirements from the domain it will actually operate in, and any non-trivial volume of work, a fine-tuned encoder of about 110 M parameters is both the more accurate and much the cheaper option. Entering an unlabelled or shifting domain changes the answer: a mid-size quantised open model gives up some in-domain accuracy, degrades far more gracefully, and runs on one consumer-class GPU without an API contract. What the results argue against is the implicit default of much recent work: fine-tuning on one corpus and assuming the result transfers.

D. Contamination and availability

Both corpora are old, small and public, so prompted in-domain scores may be inflated by pre-training contamination [27]. That bias would overstate the prompted models in-domain, which is where we report the encoders winning, so the in-domain advantage is conservative; and it cannot produce the cross-dataset reversal, because it would help them equally in both regimes. The pipeline, the splits, the prediction store and the scripts that regenerate every table and figure will be released, and scoring is a pure function of the stored outputs, so every number reproduces without re-running a model.

VI. Threats to Validity

Internal. The commercial tier ran at about 60 items per cell, where 23 of its 26 paired comparisons are inconclusive, so "never significantly beaten" partly reflects limited statistical power. Free-tier quotas caused missing-not-at-random dropout on some hosted configurations; the gate excludes the affected cells rather than imputing them. Local models ran 4-bit quantised, and each configuration used a single seed.

Construct. GPU costs are imputed from a published rental rate rather than billed, and token prices are 2026 list rates, so the ratios are more durable than the absolute values. $N^{\star}$ is a cost crossover computed regardless of the accuracy gap, and strict scoring counts format non-compliance as error, which matches deployment but differs from some prior protocols.

External. Both corpora are small, dated and public, and their security label sets differ (Section III-A), so the cross-dataset regime carries definitional as well as distributional shift, on one corpus pair. Results cover English requirements and one NFR taxonomy.

VII. Conclusion

We compared four fine-tuned encoder configurations against eight prompted LLMs on three requirements-classification tasks, holding the items, the label sets and the scoring rule fixed and varying only the distance between the data a model was fitted on and the data it was scored on. Within a corpus the encoders are ahead and are never significantly beaten; across an independently annotated corpus the ordering reverses, while the encoders remain an order of magnitude cheaper per item and repay their training after a few hundred classifications. The cause is the same in both families: a class prior acquired from somewhere other than the corpus being classified. An in-distribution score is therefore not a partial answer to the question of which approach to adopt — it can be the wrong one.

Two extensions follow from the design. Parameter-efficient fine-tuning of the LLMs, with LoRA adapters over the same 4-bit checkpoints [20], would let a prompted model be re-fitted at each regime exactly as the encoders are, separating whether prompted models transfer better because they are decoder-only or simply because they were never fitted to a source corpus; Section IV-C predicts the latter. A third independently annotated corpus such as PURE [17] would turn one cross-dataset contrast into several and let the prevalence gap be varied rather than merely observed, so that prior adjustment [25] could be evaluated as a remedy rather than left, as here, as a diagnosis.

References

[1] J. Cleland-Huang et al., "Automated classification of non-functional requirements," *Requirements Eng.*, vol. 12, no. 2, pp. 103–120, 2007.

[2] "ISO/IEC/IEEE International Standard — Systems and software engineering — Life cycle processes — Requirements engineering," in *ISO/IEC/IEEE 29148:2018(E)*, pp. 1–104, 30 Nov. 2018, doi: 10.1109/IEEESTD.2018.8559686.

[3] Z. Kurtanović and W. Maalej, "Automatically classifying functional and non-functional requirements using supervised machine learning," in *Proc. IEEE RE*, 2017, pp. 490–495.

[4] T. Hey et al., "NoRBERT: Transfer learning for requirements classification," in *Proc. IEEE RE*, 2020, pp. 169–179.

[5] W. Alhoshan, A. Ferrari, and L. Zhao, "Zero-shot learning for requirements classification: An exploratory study," *Inf. Softw. Technol.*, vol. 159, art. 107202, 2023.

[6] A. El-Hajjami, N. Fafin, and C. Salinesi, "Which AI technique is better to classify requirements? An experiment with SVM, LSTM, and ChatGPT," arXiv:2311.11547, 2024.

[7] K. Kaur and P. Kaur, "The application of AI techniques in requirements classification: A systematic mapping," *Artif. Intell. Rev.*, vol. 57, art. 57, 2024, doi: 10.1007/s10462-023-10667-1.

[8] J. Peer, Y. Mordecai, and Y. Reich, "NLP4ReF: Requirements classification and forecasting — From model-based design to large language models," in *Proc. IEEE Aerospace Conf.*, 2024, pp. 1–16.

[9] M. Binkhonain and R. Alfayaz, "Are prompts all you need? Evaluating prompt-based large language models for software requirements classification," *Requirements Eng.*, 2025.

[10] W. Alhoshan, A. Ferrari, and L. Zhao, "How effective are generative large language models in performing requirements classification?" arXiv:2504.16768, 2025.

[11] Y. Tang et al., "The few-shot dilemma: Over-prompting large language models," arXiv:2509.13196, 2025.

[12] M. Chaudhary, C. Jain, and P. R. Anish, "Exploring zero-shot app review classification with ChatGPT: Challenges and potential," arXiv:2505.04759, 2025.

[13] P. Vasudevan et al., "Requirements classification using fine-tuned transformer models: A comparative study of BERT, RoBERTa, and GPT-4o," in *Proc. ACM Southeast Conf. (ACMSE)*, 2026, pp. 199–204.

[14] O. Levy et al., "AI-assisted requirements engineering: An empirical evaluation relative to expert judgment," arXiv:2604.15222, 2026.

[15] E. Knauss et al., "Supporting requirements engineers in recognising security issues," in *Proc. REFSQ*, 2011, pp. 4–18.

[16] M. Lima et al., "Software engineering repositories: Expanding the PROMISE database," in *Proc. SBES*, 2019, pp. 427–436.

[17] A. Ferrari, G. O. Spagnolo, and S. Gnesi, "PURE: A dataset of public requirements documents," in *Proc. IEEE RE*, 2017, pp. 502–505.

[18] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[19] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[20] T. Dettmers et al., "QLoRA: Efficient finetuning of quantized LLMs," in *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 10088–10115.

[21] *ISO/IEC 25010:2011, Systems and Software Engineering — SQuaRE — System and Software Quality Models*. Geneva, Switzerland: ISO, 2011.

[22] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[23] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[24] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *J. R. Statist. Soc. B*, vol. 57, no. 1, pp. 289–300, 1995.

[25] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," *Neural Comput.*, vol. 14, no. 1, pp. 21–41, 2002.

[26] Z. Zhao et al., "Calibrate before use: Improving few-shot performance of language models," in *Proc. ICML*, 2021, pp. 12697–12706.

[27] O. Sainz et al., "NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark," in *Findings of EMNLP*, 2023, pp. 10776–10787.